You said:

java application deployment in AWS setting and configuration

ChatGPT said:

Java Application Deployment on AWS — Setup and Configuration

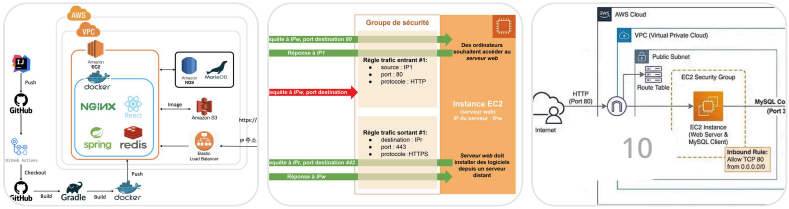
For a Java enterprise application, such as a **Spring Boot REST API, banking application, or multithreaded server**, you can deploy it on AWS without managing physical hardware.

A practical starting point is:

Java + Spring Boot + Maven + AWS EC2 + Ubuntu + Nginx + PostgreSQL

I'll explain the deployment architecture, AWS settings, server configuration, and the steps to deploy a working Java application.

1. AWS deployment architecture



The application follows this request flow:

```
User / Postman
      |
   HTTPS
      |
      v
Nginx Reverse Proxy
      |
      v
Java Spring Boot
  Port 8080
      |
      v
PostgreSQL DB
```

Component	Purpose
Java 17 or 21	Runs your Java application
Spring Boot	Provides REST APIs and application services
Maven	Builds the application into a JAR
AWS EC2	Virtual machine that runs your application
Ubuntu Server	Operating system on EC2
Nginx	Reverse proxy and HTTPS termination
PostgreSQL	Stores application data
AWS Security Groups	Controls network access
CloudWatch	Collects logs and monitors server metrics
AWS IAM	Controls permissions for AWS resources

Important: You do not need every component on day one. Start with EC2 and your Java JAR, then add Nginx, a database, HTTPS, and monitoring.

2. Configure AWS EC2

Open the official AWS Management Console.

Step 1: Launch a virtual server

Navigate to **EC2 → Instances → Launch instances**.

Use these settings:

Setting	Recommended configuration
Name	`java-backend-server`
Operating system	Ubuntu Server LTS
Architecture	64-bit x86
Instance type	A small instance for initial testing; increase resources if needed
Key pair	Create a new key pair and download the private key

Setting	Recommended configuration
VPC	Default VPC for a simple learning deployment
Subnet	Public subnet for this initial setup
Auto-assign public IP	Enable
Storage	Start with the default size and increase as needed

For Java, select an instance with sufficient memory for the JVM, operating system, and application. A small instance may work for a simple API, but a database or several services may require more RAM.

Follow the official EC2 instance launch guide.

Step 2: Configure the Security Group

A security group acts as a virtual firewall.

Configure these inbound rules:

Type	Port	Source	Purpose
SSH	22	Your IP address	Remote server administration
HTTP	80	`0.0.0.0/0`	Public HTTP access
HTTPS	443	`0.0.0.0/0`	Public HTTPS access

Security recommendations:

- Never expose SSH port 22 to the entire internet unless temporarily required for a controlled test.
- Do not expose PostgreSQL port 5432 publicly.
- Do not expose Java port 8080 publicly in the final Nginx-based configuration.
- Add IPv6 rules if you intend to provide public access over IPv6.

See the official AWS Security Group documentation.

Step 3: Connect to EC2

Once the instance is running, connect using SSH from your Manjaro terminal.

Set the private key permissions:

```
chmod 400 java-backend-key.pem
```

Connect to your instance:

```
ssh -i java-backend-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Replace `YOUR_EC2_PUBLIC_IP` with the instance's actual public IP address.

For more information, see AWS SSH connection instructions.

---

3. Install Java on the AWS server

After connecting to Ubuntu, update the package list:

```
sudo apt update
sudo apt upgrade -y
```

Install Java 21:

```
sudo apt install openjdk-21-jdk -y
```

Verify the installation:

```
java -version
javac -version
```

Check the Java executable location:

```
which java
```

Check the `JAVA_HOME` value:

```
readlink -f "$(which java)"
```

If your project uses Java 17, install the corresponding JDK instead. The Java version on the server should be compatible with the version used to build your application.

Note: Package availability depends on the Ubuntu release and configured repositories.

4. Build your Java application

Suppose you have a Spring Boot application with a REST endpoint:

```
GET /api/hello
```

Example response:

```
{
  "message": "Hello from AWS"
}
```

Build the application on your development machine.

For a Maven project:

```
mvn clean package
```

Your executable JAR will usually be generated in:

```
target/myapp-0.0.1-SNAPSHOT.jar
```

The exact filename depends on your project configuration.

Transfer the JAR to AWS

From your local machine, run:

```
scp -i java-backend-key.pem \
    target/myapp-0.0.1-SNAPSHOT.jar \
    ubuntu@YOUR_EC2_PUBLIC_IP:/home/ubuntu/
```

This copies the application to the server.

Alternatively, you can build the application directly on EC2, but building locally and transferring the JAR is a straightforward deployment workflow.

5. Run the Java application

On the EC2 server, start the application:

```
java -jar myapp-0.0.1-SNAPSHOT.jar
```

By default, a typical Spring Boot web application listens on port 8080.

Test the API locally on the EC2 server:

```
curl http://localhost:8080/api/hello
```

If the endpoint is working, you should receive the JSON response.

You can also test it from Postman, but if port 8080 is not open in the security group, direct public access will not work. That is intentional in the recommended final configuration.

For an initial test only, you could temporarily allow port 8080 from your own IP address. Remove that rule after configuring Nginx.

---

6. Configure the application as a system service

Running Java directly in a terminal is inconvenient because the process stops when you terminate it.

Use `systemd` to manage the application.

Step 1: Create an application directory

```
sudo mkdir -p /opt/javaapp
sudo cp /home/ubuntu/myapp-0.0.1-SNAPSHOT.jar /opt/javaapp/app.jar
sudo chown -R ubuntu:ubuntu /opt/javaapp
```

Step 2: Create a service file

```
sudo nano /etc/systemd/system/javaapp.service
```

Add the following configuration:

```
[Unit]
Description=Java Spring Boot Application
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/opt/javaapp
ExecStart=/usr/bin/java -jar /opt/javaapp/app.jar
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Save the file.

Step 3: Enable and start the service

```
sudo systemctl daemon-reload
sudo systemctl enable javaapp
sudo systemctl start javaapp
```

Check its status:

```
sudo systemctl status javaapp
```

View application logs:

```
sudo journalctl -u javaapp -f
```

Now the application can start automatically when the server boots and restart after certain failures.

For a production application, also configure appropriate JVM memory limits, application configuration, and a dedicated service account with only the permissions it needs.

---

7. Configure Nginx as a reverse proxy

The goal is to allow users to access your application using HTTP or HTTPS without exposing the Java server's port directly.

Install Nginx:

```
sudo apt install nginx -y
```

Create a configuration file:

```
sudo nano /etc/nginx/sites-available/javaapp
```

Add:

```
server {
    listen 80;
    server_name YOUR_DOMAIN_OR_PUBLIC_IP;

    location / {
        proxy_pass http://127.0.0.1:8080;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Replace `YOUR_DOMAIN_OR_PUBLIC_IP` with your domain name or public IP address.

Enable the configuration:

```
sudo ln -s /etc/nginx/sites-available/javaapp \
    /etc/nginx/sites-enabled/javaapp
```

Test the configuration:

```
sudo nginx -t
```

Restart Nginx:

```
sudo systemctl enable nginx
sudo systemctl restart nginx
```

Test the endpoint:

```
curl http://YOUR_EC2_PUBLIC_IP/api/hello
```

The request now follows this route:

```
Postman
   |
   | HTTP :80
   v
Nginx
   |
   | localhost:8080
   v
Spring Boot REST API
```

Once this works, restrict access to port 8080 so that only local services can reach it.

Enable HTTPS

For a public production application, configure a domain name, point its DNS record to your server, and obtain a TLS certificate.

You can use Let's Encrypt with Certbot and configure Nginx to serve HTTPS on port 443. The HTTP configuration above is only the starting point; it does not enable HTTPS by itself.

---

8. Configure a database

For an enterprise Java application, the database should generally be separate from the application process.

A suitable AWS setup is:

Component	AWS service
Java application	EC2
Relational database	Amazon RDS
Database engine	PostgreSQL
Application configuration	Environment variables or managed configuration
Database credentials	AWS Secrets Manager

For example, your Spring Boot configuration could use environment variables:

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

Configure the actual database URL, username, and password on the server rather than committing credentials to Git.

For a secure deployment:

1. Place the RDS database in a private subnet.
2. Configure its security group to accept database traffic only from the application's security group.
3. Use a managed secrets service or another secure credential-management approach.
4. Enable backups and configure retention appropriate to your requirements.

For learning, you can initially use a local database or a lightweight database. For production, a managed database reduces the work involved in database maintenance and backups.

---

9. Configure logging and monitoring

Use Amazon CloudWatch to monitor your application environment.

Monitor:

- EC2 CPU utilization.
- Available memory using the CloudWatch Agent.
- Disk usage.
- Application errors and exceptions.
- Request latency and API availability.
- Database connections and performance.

For initial debugging, use:

```
sudo journalctl -u javaapp -f
```

For AWS-level monitoring, assign an appropriate IAM role to EC2 rather than storing permanent AWS access keys in your application.

See the official CloudWatch Agent setup documentation and EC2 IAM role documentation.

---

10. AWS configuration checklist

Before considering the deployment complete, verify these settings.

Area	Required configuration
Compute	EC2 instance running a supported operating system
Java	Compatible JDK/runtime installed

Area	Required configuration
Application	Executable JAR starts successfully
Service management	`systemd` service enabled
Network	Correct VPC, subnet, routes, and security groups
Public access	Nginx handles incoming web requests
HTTPS	Valid TLS certificate and domain configuration
Database	Private connectivity, backups, and secure credentials
Permissions	IAM roles with least-privilege permissions
Monitoring	CloudWatch metrics and application logs
Deployment	Repeatable build and deployment procedure
Recovery	Tested backups and recovery procedure

What about costs?

EC2, storage, public IPv4 addresses, data transfer, databases, and monitoring can incur charges. AWS free-tier eligibility and allowances depend on the account and current offer.

Before launching resources, check the current pricing and available free-tier benefits on the official AWS website and set a billing budget or alert. A budget alert does not automatically stop resources from generating charges.

---

11. Recommended learning path

For a Java backend developer, I recommend learning these deployment stages in order:

1. **EC2:** Launch a server and connect over SSH.
2. **Linux:** Install Java and run a JAR.
3. **systemd:** Keep the Java application running as a service.
4. **Nginx:** Configure reverse proxying.
5. **HTTPS:** Configure a domain and TLS.
6. **PostgreSQL/RDS:** Connect the application to a database.
7. **IAM and Secrets Manager:** Configure permissions and credentials.
8. **CloudWatch:** Monitor the application.
9. **Docker:** Package the application into a container.

10. **CI/CD:** Automate building and deployment with a pipeline.

Once you understand this setup, you can progress to load balancing, multiple EC2 instances, Auto Scaling, and more advanced architectures.

Which deployment approach do you want to learn?

A. Basic deployment: Java JAR → EC2 → Ubuntu → run the application.

B. Enterprise deployment: Spring Boot → EC2 → Nginx → HTTPS → PostgreSQL/RDS → CloudWatch.

C. Modern cloud deployment: Spring Boot → Docker → AWS ECS/Fargate → RDS → load balancer → CI/CD.

My recommendation: Start with **B**. It teaches the server configuration, networking, security, and database integration commonly encountered when deploying Java backend applications.